COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	7/7/26	**DATE SUBMITTED:**	7/13/26
NAME:	Vince Andrew Urbano	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1 **Basic C++ Programming**

1. PROCEDURE OUTPUT

ILO-A:
Code:

CMakeLists.txt main.cpp

```cpp
#include<iostream>

int sumOfTwoNum(int fNum, int sNum);
void greaterValue(int fNum, int sNum);
bool booleanValues(bool fNum, bool sNum);
int main(){
    std::cout<<"The sum of two numbers is: "<<sumOfTwoNum( fNum: 6, sNum: 7)<<std::endl;
    greaterValue( fNum: 6, sNum: 7);
    booleanValues( fNum: 0, sNum: 1);
}
int sumOfTwoNum(int fNum, int sNum){
    int sum = fNum +sNum;
    return sum;
}
void greaterValue(int fNum, int sNum){
    if (fNum > sNum){
        std::cout<< fNum << " is greater than " << sNum << std::endl;
    }
    else{
        std::cout<< sNum << " is greater than " << fNum << std::endl;
    }
}
```

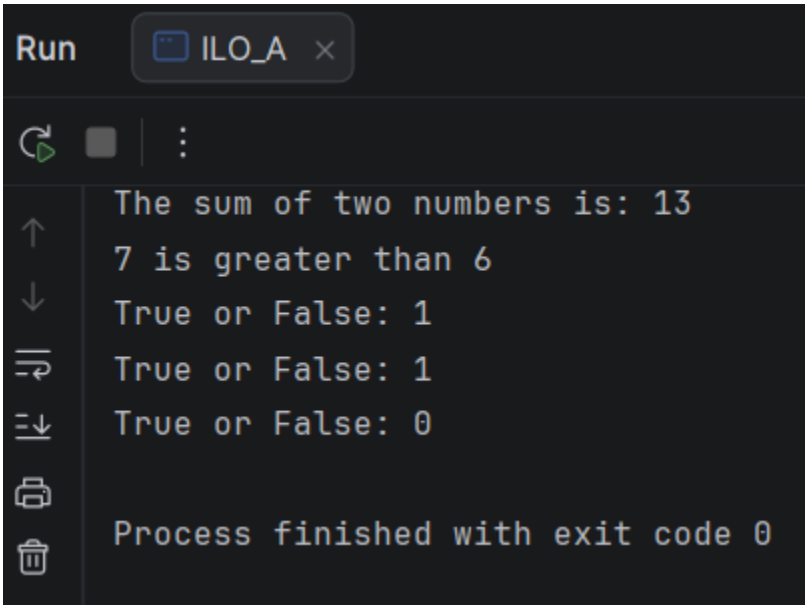
```cpp
bool booleanValues(bool fNum, bool sNum){
    bool AND = fNum&&sNum;
    bool OR = fNum ||sNum;
    bool NOT = !fNum;
    bool NOT2 = !sNum;

    std::cout<< "True or False: " << OR << std::endl;
    std::cout<< "True or False: " << NOT << std::endl;
    std::cout<< "True or False: " << AND << std::endl;

    return true;
}
```

Code Analysis:
In this code, we have created three functions named sumOfTwoNum, greaterValues, and booleanValues. The function sumOfTwoNum only accepts integers, it is responsible for getting the sum of two different values. On the other hand, greaterValues checks the values and determines which number is greater than the other. Lastly, booleanValues is responsible for checking the logical operators.

Output:



ILO-B:

Section	Answer
Header File and Declaration	`#include <iostream>`
Global Declaration Section	`int count = 0;`
Class Declaration and Method Definition Section	`class Triangle{ private: double totalAngle, angleA, angleB, angleC; public: Triangle(double A, double B, double C); void setAngles(double A, double B, double C); const bool validateTriangle(); };`
Main Function	`int main() { Triangle set1( A: 40, B: 30, C: 110); if (set1.validateTriangle()) { std::cout << "The shape is a valid triangle.\n"; }else { std::cout << "The shape is Not a valid triangle.\n"; } return 0; }`
Method Definition	`Triangle::Triangle(double A, double B, double C) { angleA = A; angleB = B; angleC = C; totalAngle = A+B+C; } void Triangle::setAngles(double A, double B, double C) { angleA = A; angleB = B; angleC = C; } const bool Triangle::validateTriangle() { return (totalAngle <=180); }`

Code:

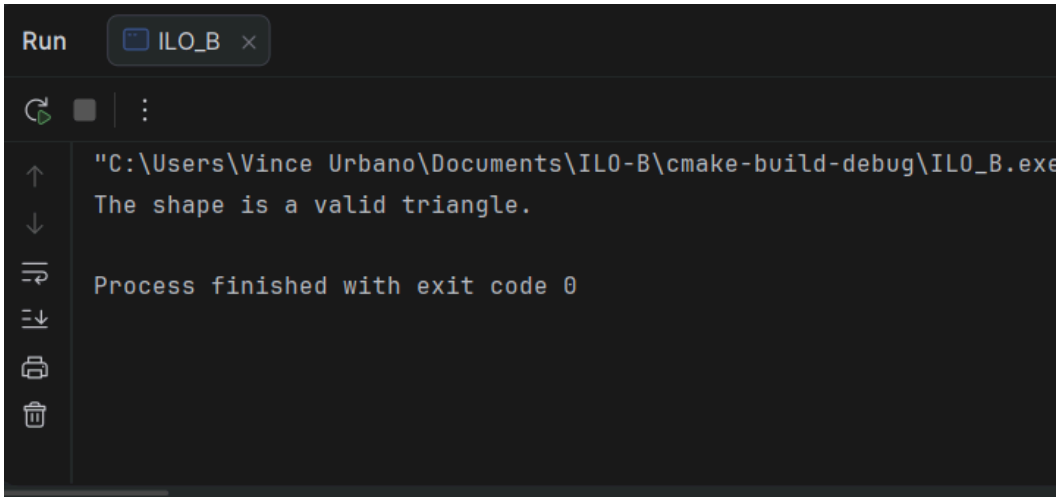
```cpp
#include <iostream>

class Triangle{
private:
    double totalAngle, angleA, angleB, angleC;
    public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
    };
int count = 0;
int main() {
    Triangle set1( A: 40, B: 30, C: 110);
    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";
    }else {
        std::cout << "The shape is Not a valid triangle.\n";
    }
    return 0;
}
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
```

```
27  void Triangle::setAngles(double A, double B, double C) {
28      angleA = A;
29      angleB = B;
30      angleC = C;
31  }
32  const bool Triangle::validateTriangle() {
33      return (totalAngle <=180);
34  }
35
```

Code Analysis:
A class is created named Triangle, in this class there are different functions. Each function will do their assigned functions. The triangle function is responsible for getting the total angles. While the setAngles are responsible for assigning a value for each angle such as, angleA, angleB and angleC. Lastly const bool will be responsible to check if the triangle is a valid triangle.

Output:



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. Create a C++ program to swap the two numbers in different variables.

Code:

CMakeLists.txt | main.cpp

```cpp
#include <iostream>
//Supplementary 1
void swapped(int num1, int num2) {
    int Swap = num1;
    num1 = num2;
    num2 = Swap;
    std::cout << "Numbers Swapped! \n" << num1 << " " << num2 << std::endl;

}

int main() {
    swapped(23, 24);

}
```

Code Analysis:

A function named swapped is created, it will swap the position of the values which were assigned in int main(). The value of num1 will be assigned to the Swap and the num2 will be assigned to num1, and the value of Swap will be assigned to num2, therefore the positions of the values are swapped.

Output:

Run | ILO_C

```
"C:\Users\Vince Urbano\Documents\ILO-C\cmake-build-debug\ILO_C.exe"
Numbers Swapped!
24 23

Process finished with exit code 0
```

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.
Code:

CMakeLists.txt | main.cpp

```cpp
#include <iostream>
//Supplementary 2

void convert(double K) {
    double F = (K - 273.15)*(1.8)+32;
    std::cout<< K <<" Kelvin in Fahrenheit is " << F <<std::endl;
}
int main() {
    double T;

    std::cout << "Enter temperature in Kelvin: " ;
    std::cin >> T;
    convert(T);
}
```

Code Analysis:
In this code, a function has been made to convert Kelvin to Fahrenheit. The user will input a value in Kelvin which will then be converted to Fahrenheit.

Output:

Run | Supplemenatry_2

```
"C:\Users\Vince Urbano\Documents\Supplementary\Supplemenatry 2\cmake-build-debug\Supplemenatry_2.exe"
Enter temperature in Kelvin:23

23 Kelvin in Fahrenheit is -418.27

Process finished with exit code 0
```

3. Create a C++ program that has a function that will calculate the distance between two points.

CMakeLists.txt | main.cpp

```cpp
#include <iostream>
#include <cmath>
//Supplementary 3
void distance(int xA, int xB, int yA, int yB) {
    double d = std::pow(x: (xB - xA), y: 2) + std::pow(x: yB - yA, y: 2) ;
    double d1 = sqrt(d);
    std::cout << "The distance between the points, "<< xA << ", " << yA << " and " << xB <<", "<< yB << " is " << d1 << std::endl;
}
int main() {
    int xA, xB, yA, yB;
    std::cout << "Enter the point x1: ";
    std::cin >> xA;
    std::cout << "Enter the point x2: ";
    std::cin >> xB;
    std::cout << "Enter the point y1: ";
    std::cin >> yA;
    std::cout << "Enter the point y2: ";
    std::cin >> yB;
    distance(xA, xB, yA, yB);
}
```

Code Analysis:
In this code, I have imported cmath in order to simplify my codes. Instead of using a longer code just to find the square root of a value, I can just use the sqrt() function. With the use of cmath, I also used the pow() function which will be used to square the values. This function will be responsible for getting the distance between two different points.

Output:

Run | Supplementary_3

```
"C:\Users\Vince Urbano\Documents\Supplementary\Supplementary 3\cmake-build-debug\Supplementary_3.exe
Enter the point x1:1

Enter the point x2:5

Enter the point y1:3

Enter the point y2:6

The distance between the points, 1, 3 and 5, 6 is 5

Process finished with exit code 0
```

4.Modify the code given in ILO B and add the following functions:
a. A function to compute for the area of a triangle
Code:

CMakeLists.txt | main.cpp

```cpp
#include <iostream>

class Triangle{
private:
    double totalAngle, angleA, angleB, angleC;
    public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
    double areaTriangle(double base, double height);
    };

int count = 0;
int main() {
    Triangle set1( A: 40, B: 30, C: 110);
    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";
    }else {
        std::cout << "The shape is Not a valid triangle.\n";
    }
    set1.areaTriangle( base: 40, height: 110);
    return 0;
}
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
```

```
    totalAngle = A+B+C;
}
void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
}
const bool Triangle::validateTriangle() {
    return (totalAngle <=180);
}
double Triangle::areaTriangle(double base, double height) {
    double area =  (base*height)/2;
    std::cout << "The area of your triangle is " << area << "\n";
    return area;
}
```

Output:

```
Run  Supplementary_4
"C:\Users\Vince Urbano\Documents\Supplementary\Supplementary 4\cmake-build-debug\Supplementary_4.exe"
The shape is a valid triangle.
The area of your triangle is 2200

Process finished with exit code 0
```

b. A function to compute for the perimeter of a triangle

CMakeLists.txt | main.cpp

```cpp
#include <iostream>

class Triangle{
private:
    double totalAngle, angleA, angleB, angleC;
    public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
    double areaTriangle(double base, double height);
    double perimeterTriangle(double A, double B, double C);
    };

int count = 0;
int main() {
    Triangle set1( A: 40, B: 30, C: 110);
    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";
    }else {
        std::cout << "The shape is Not a valid triangle.\n";
    }
    set1.areaTriangle( base: 40, height: 110);
    set1.perimeterTriangle( A: 12, B: 10,  C: 20);
    return 0;
}
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
```

```
30      totalAngle = A+B+C;
31  }
32  void Triangle::setAngles(double A, double B, double C) {
33      angleA = A;
34      angleB = B;
35      angleC = C;
36  }
37  const bool Triangle::validateTriangle() {
38      return (totalAngle <=180);
39  }
40  double Triangle::areaTriangle(double base, double height) {
41      double area =  (base*height)/2;
42      std::cout << "The area of your triangle is " << area << "\n";
43      return area;
44  }
45  double Triangle::perimeterTriangle(double A, double B, double C) {
46      double P = A+B+C;
47      std::cout << "The perimeter of your triangle is " << P;
48      return P;
49  }
```

Output:

```
Run  Supplementary_4
"C:\Users\Vince Urbano\Documents\Supplementary\Supplementary 4\cmake-build-debug\Supplementary_4.exe"
The shape is a valid triangle.
The area of your triangle is 2200
The perimeter of your triangle is 42
Process finished with exit code 0
```

c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

Code:

CMakeLists.txt main.cpp

```cpp
#include <iostream>

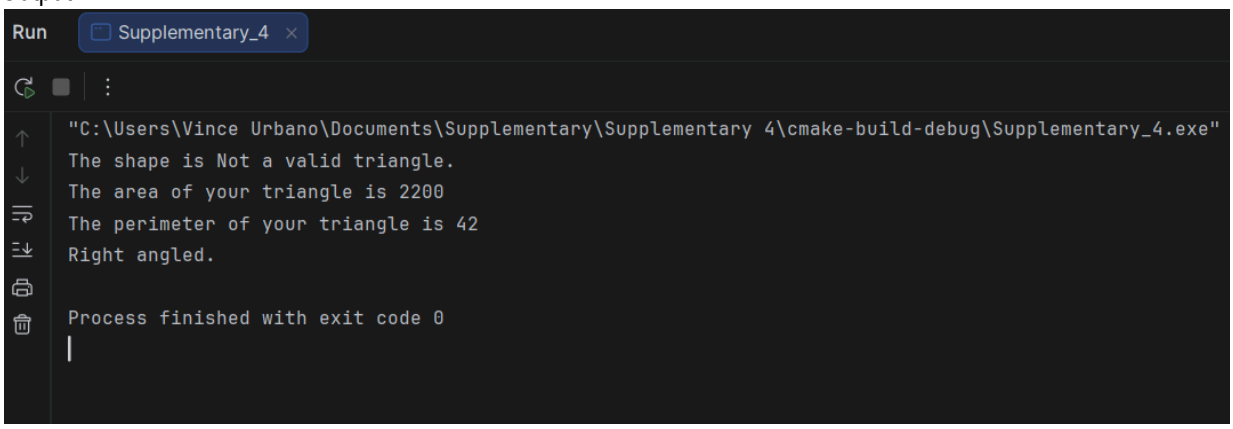
class Triangle{
private:
    double totalAngle, angleA, angleB, angleC;
    public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
    double areaTriangle(double base, double height);
    double perimeterTriangle(double A, double B, double C);
    void triangleType();
    };

int count = 0;
int main() {
    Triangle set1( A: 80, B: 190, C: 90);
    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";
    }else {
        std::cout << "The shape is Not a valid triangle.\n";
    }
    set1.areaTriangle( base: 40, height: 110);
    set1.perimeterTriangle( A: 12, B: 10,  C: 20);
    set1.triangleType();
    return 0;
}
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
```

CMakeLists.txt | main.cpp

```cpp
Triangle::Triangle(double A, double B, double C) {
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
}
const bool Triangle::validateTriangle() {
    return (totalAngle <=180);
}
double Triangle::areaTriangle(double base, double height) {
    double area =  (base*height)/2;
    std::cout << "The area of your triangle is " << area << "\n";
    return area;
}
double Triangle::perimeterTriangle(double A, double B, double C) {
    double P = A+B+C;
    std::cout << "The perimeter of your triangle is " << P<< std::endl
    return P;
}
void Triangle::triangleType() {
    if (angleA == 90 or angleB == 90 or angleC == 90) {
        std::cout << "Right angled.\n";
    }
    else if (angleA > 90 or angleB > 90 or angleC > 90){
        std::cout << "Obtuse angled.\n";
    } else{
        std::cout << "Acute angled\n";
    }
}
```

Output:



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini, ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance and explanation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used the AI softwares to ask how to import and use the functions from that library.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.